Och

1 Introduction

Static analysis catches bugs before software is deployed, notifying developers of mistakes earlier in the development pipeline than testing or user reports can. Type systems are the most widely adopted form of static analysis, and the strength of guarantees they provide varies enormously: from freedom from runtime type errors (TypeScript) through ownership-based memory safety (Rust) to full functional correctness (Lean, Agda, Coq). At the strong end of this spectrum lies *formal verification*: a compiler-checked mathematical proof that a program satisfies its specification across all possible inputs.

Formal verification has been applied at scale: a verified C compiler (Leroy 2009), a verified operating system kernel (Klein et al. 2009), and a verified HTTPS stack (Bhargavan et al. 2017) all demonstrate that proving the correctness of real systems software is feasible. These projects used a variety of proof technologies, but a common thread among the most ergonomic approaches is *dependent types*: a type-theoretic mechanism by which types may depend on values, allowing the programmer to state properties such as "this function returns a list of the same length as its input" directly in a type signature, with the compiler verifying the property as a condition of successful compilation.

There is a large intersection between i) software which requires low-level control and performance, and ii) software where correctness is critical. Operating system kernels, cryptographic libraries, network protocol implementations, and compiler backends all occupy this intersection — they are *systems software*. Today, the languages which offer the necessary control (C, C++, Rust) lack the type-theoretic machinery for formal verification, while the languages which support dependent types (Lean, Agda, F*) lack the control over memory layout, allocation, and mutation that systems programming demands.

Ochre is a planned language which aims to inhabit this gap: a systems programming language with native support for dependent types. One might summarise the goal as combining Rust's ownership model and low-level control with Lean's proof capabilities in a single language and type system. The key mechanism that enables Ochre is the isomorphism between (safe) Rust and Lean (Ho and Protzenko 2022) — if we can translate freely between Rust and Lean, and we know how to dependently type check Lean, then we must be able to dependently type check Rust.

Two recent trends make this research direction timely. First, the cost of unverified software is rising: AI-assisted vulnerability discovery has dramatically increased the rate at which security bugs are found in critical infrastructure — Mozilla closed as many Firefox vulnerabilities in April 2026 as in the previous two years combined (Knop 2026), and a nine-year dormant Linux kernel privilege escalation was surfaced by automated analysis (Microsoft Security Response Center 2026). Second, the cost of writing software is falling: AI coding agents have reduced the labour involved in software engineering to the point where ambitious rewrites that were previously infeasible are now routine (Sumner 2026). The bottleneck for software engineering is shifting from writing code to specifying what correct code looks like; the value proposition of a language whose type system *is* a specification language is growing stronger.

During a previous attempt to develop Ochre (Lidbury 2024), the project was held back by soundness bugs in the type system and a general lack of clarity on the semantics of the objects in the language. A counterexample was found in which narrowing a variable's type — the operation which underlies strong mutation — invalidated a previously valid subtyping judgment. The root cause was not in the mutation or ownership machinery, but in the foundational interaction between subtyping and type-level computation: the core semantic idea that terms and types share a common language had not been studied carefully enough to support the full system built on top of it.

This paper presents *Och*, a core calculus designed to isolate and study these foundational features before reintroducing the complexity of mutation and ownership. Och combines λ-calculus with equirecursive subtyping, self-types (ι), and general recursion (fix) in a setting where terms and types are syntactically and semantically unified — concretely, there is no separate Π-type constructor for function types; the type of a function is itself a function, and types reduce by the same β-rule as terms. Och is the first

in a planned sequence of calculi — Och, Ochr (adding ownership/linearity), and finally Ochre (adding mutable references) — each of which answers a distinct research question while building toward the full language.

1.1 Methodology

Och has been designed experimentally: a set of *goal programs* drove the design of the typing rules. When a candidate rule set accepted all current goal programs and rejected their deliberately buggy variants, new and more demanding goal programs were added and the process iterated. This methodology has two consequences.

First, Och does not yet rest on a unifying semantic theory, and attempts at a soundness proof are ongoing. Establishing such a theory is a natural next step now that the rule set has stabilised around a substantial body of examples.

Second, the resulting test suite (see Section 5) provides strong evidence of expressivity. Using Church/Scott encodings, Och expresses booleans, natural numbers, finite sets ($\mathsf{Fin}\ n$), pairs, length-indexed arrays, and safe indexing — all with dependent elimination. The type system catches bugs as subtle as adding the *wrong* two numbers (see Section 5.7), not merely checking that an addition is well-shaped.

1.2 Goals

Soundness. Ochre is only valuable if programmers can trust the compiler to catch runtime errors. Proving the soundness of a system which combines equirecursive subtyping, self-types, and general recursion in a unified term/type language is, as far as I can tell, novel. Och is the setting in which to carry out this research.

Expressivity. The type system must articulate properties and catch bugs that programmers care about. It must be expressive enough to distinguish $x + y$ from $x + x$ when one is correct and the other is not.

Extensibility. Och is the first in a series of calculi building toward Ochre. Features which do not generalise to the full language are not worth investigating here. This consideration has already manifested in the decision to use general recursion via fix rather than adding primitive inductive types with only well-founded induction.

2 Background

These are works which Och builds off directly, and are worth understanding on at least a surface level.

§1-3 of Hutchins (2010) is a particularly good read. It's a 12 page compression of his 336 page thesis and is surprisingly self contained considering how novel the concepts presented are. §4-8 are not needed at all to understand Och, since they focus primarily on results and proofs.

2.1 Pure Subtype Systems

In a conventional type theory, there is a sharp syntactic distinction between terms and their types. Functions are introduced by λ and their types by Π: the term $\lambda(x : A).b$ has type $\Pi(x : A).B$. These are different constructors with different reduction rules — λ participates in β-reduction, while Π does not compute.

Pure Subtype Systems (Hutchins 2010) collapse this distinction. There is only λ: the type of a function *is* a function. Where a conventional system writes $\Pi(x : A).B$ for the type of functions from A to B, PSS writes $\lambda(x : A).B$ — the same constructor used for the functions themselves. This means types compute by exactly the same rules as terms: β-reduction, substitution, and application all apply uniformly.

The consequence is that every term is trivially its own most precise type. A value like 3 does not merely *have* type Nat; it *is* a type, the singleton type whose only inhabitant is 3. The subtyping relation then does all the work that a typing judgment would normally do: $3 \sqsubseteq \mathsf{Nat}$ is the statement that 3 is a natural number. There is no need for a separate $\Gamma \vdash e : \tau$ judgment — well-formedness (Section 4.2) checks structural validity, and subtyping (Section 4.4) answers every question about type compatibility.

Och adopts PSS's core design and extends it with self-types (ι), general recursion (fix), and coinductive subtyping.

2.2 Self Types

Church encodings represent data as their own eliminators: a boolean is a function that takes two branches and returns one of them, a natural number is a function that takes a zero case and a successor case and folds over itself. This is elegant but has a well-known limitation: the eliminator's return type cannot depend on the value being eliminated. A Church-encoded boolean can branch on itself to return values of some fixed type X, but it cannot branch to return values of type P true or P false for a type-level function P — the eliminator does not know which boolean it is.

Self types, introduced by (Fu and Stump 2014), solve this problem. The binder $\iota(x \leq \tau).e$ introduces a type in which the variable x refers to *the term inhabiting the type*. When checking whether $a \sqsubseteq \iota(x \leq \tau).e$, the rule [S-Iota-Intro] substitutes a for x in the body, reducing the goal to $a \sqsubseteq e[x \mapsto a]$. This feeds the value being typed *into* its own type, enabling the return type to depend on the inhabitant. Self types are also used by Kind (Taelin 2021a), a (now-defunct) proof language by Victor Maia (AKA Victor Taelin online), which provides an accessible introduction to the idea.

For example, the dependent boolean type DBool is defined so that its eliminator takes a motive P : DBool $\to \top$ and branches of type P true and P false, returning $\boldsymbol{P}$ **self** where self is the boolean value currently being typed. The constructors true and false are plain λ-terms, identical to their non-dependent counterparts — the dependent elimination machinery lives entirely in the type, not in the constructors. See Section 5.1 for the full definitions and Section 5 for further encodings built on this pattern.

3 Language Semantics

This section describes what each piece of the language does in natural language to give the reader an intuition for the objects involved before we get into the typing rules, examples, and metatheory. The full syntax is given in Figure 1. When the annotation τ in a binder is $\top$, we omit it: $\lambda x.e$ abbreviates $\lambda(x \leq \top).e$, and similarly for ι and fix.

$$
\begin{array}{rr}
e, \tau ::= x & \text{variable} \\
\mid \lambda(x \leq \tau).e & \text{lambda} \\
\mid e_1\ e_2 & \text{application} \\
\mid \top & \text{universe (top)} \\
\mid \bot & \text{primitive bottom} \\
\mid \iota(x \leq \tau).e & \text{self-type binder} \\
\mid \text{fix}(x \leq \tau).e & \text{recursive binder}
\end{array}
$$

Figure 1: Och syntax.

Variables, lambdas, application as per the standard λ-calculus, except λ has a domain type annotation which is respected by the type checker.

$\top$ **and** $\bot$ are the top and bottom of the subtyping lattice. $\top$ represents the widest possible type, which tells you nothing about the term being typed, and $\bot$ is the empty type.

Fix $\text{fix}(x \leq \tau).e$ allows a term to be defined in terms of itself. x is the reference to the whole fix expression itself, and τ is the "upper bound" which the whole expression can be assumed to have while it's being type checked (which prevents loops during type checking).

Iota $\iota(x \leq \tau).e$ allows the definition of a type to include a reference to *the term inhabiting it*. This means $e \sqsubseteq \iota(x : \tau_0).\tau_1$ is reduced to $e \sqsubseteq \tau_1[x \mapsto e]$ during checking (notice: the LHS has moved to the RHS). This is a Cedille-style self type (Fu and Stump 2014), and allows for church encodings with dependent elimination instead of having to add inductive datatypes to the language as a primitive. See Section 2.2 for a longer explanation of self types.

3.1 Concrete evaluation

The concrete evaluator defines what it means for a closed term to produce a value at runtime. It plays a crucial role in the soundness statement: soundness asserts "if a program is well-formed, it will not get stuck during evaluation." For this to be non-vacuous, the evaluator must be *partial* — it must reject ill-formed programs by having no applicable rule, rather than silently succeeding on all inputs.

Concrete evaluation is a substitution-based, call-by-value, big-step semantics on closed terms. The judgment $e \Downarrow v$ states that closed term e evaluates to value v. There is no context and no free variables: binders are eliminated by eager substitution. The rules are given in Figure 2.

The value forms are $\{\lambda, \top, \bot\}$. [E-Val] states that values evaluate to themselves. [E-App] evaluates the function to a λ, evaluates the argument, substitutes, and evaluates the body. [E-Iota] and [E-Fix] eagerly unroll by substituting the binder's self-reference into the body and evaluating the result.

As a worked example, evaluating the application $(\lambda(x \leq \top).x)\ \top$:

$$(\lambda(x \leq \top).x)\ \top \Downarrow \top$$

by [E-App]: the function $(\lambda(x \leq \top).x)$ is already a value, the argument $\top$ is a value by [E-Val], and the substituted body $x[x \mapsto \top] = \top$ evaluates to $\top$ by [E-Val].

Aside: Och has no concept of levels or universes, so the type checker cannot enforce that $\{\top, \bot, \iota\}$ do not appear at runtime; the evaluator must handle them gracefully. Adding a universe hierarchy (see §3.6 of PSS (Hutchins 2010)) would allow type-level arguments to be erased during compilation, after which $\{\top, \bot, \iota\}$ would never appear at level 0. I plan on doing this in the future.

$$\frac{}{v \Downarrow v}\ \text{E-Val} \qquad \frac{f \Downarrow \lambda(x \leq \tau).b \quad a \Downarrow v_a \quad b[x \mapsto v_a] \Downarrow v}{f\ a \Downarrow v}\ \text{E-App}$$

$$\frac{b[x \mapsto \iota(x \leq \tau).b] \Downarrow v}{\iota(x \leq \tau).b \Downarrow v}\ \text{E-Iota} \qquad \frac{b[x \mapsto \text{fix}(x \leq \tau).b] \Downarrow v}{\text{fix}(x \leq \tau).b \Downarrow v}\ \text{E-Fix}$$

$$\begin{aligned} \text{where } v \in \text{Value} ::= {} & \lambda(x \leq \tau).e && \text{lambda} \\ \mid {} & \bot && \text{primitive bottom} \\ \mid {} & \top && \text{universe (top)} \end{aligned}$$

Figure 2: Evaluation rules. Big-step, substitution-based, call-by-value on closed terms.

The most important aspect of the evaluation rules is what is *absent*:

- Free variables are not values and have no evaluation rule, so they cannot occur at runtime. This is what forces the type system to rule out ill-scoped variables.
- [E-App] requires the function position to evaluate to a λ. Applying $\top$, $\bot$, or any non-function term is stuck — there is no rule for it. This is what forces the type system to verify that the function position has an appropriate type before allowing an application.

A term like $\text{fix}(x \leq \top).x$ unrolls indefinitely via [E-Fix]: the body x is replaced by the whole expression, producing the same term again. This is the correct behaviour for non-terminating recursion.

4 Typing Rules

Och's type system is structured around two judgements:

Well-formedness $\Gamma \vdash e$ wf states "e is well-formed under Γ" and is the "entry point" for the type system. Defined in Section 4.2. Delegates all type-comparison questions to subtyping.

Subtyping $S; \Gamma \vdash a \sqsubseteq b$ states "a is a subtype of b under Γ, assuming coinductive hypotheses S". Subtyping in Och means set inclusion: $A \sqsubseteq B$ iff every value of A is also a value of B. Section 4.3 explains the S coinductive hypothesis system.

4.1 Context Γ

Γ is the typing context defined as $\Gamma ::= \emptyset \mid \Gamma, x \leq a$. Lookup is $\Gamma(x)$. Each entry describes what the type checker knows about a variable statically.

4.2 Well-formedness

The well-formedness judgment $\Gamma \vdash e$ wf determines whether term e is well-formed under context Γ. In other systems this role would be served by a $\Gamma \vdash a : \tau$ rule which assigns a type to a term, but there is no need to assign a type to terms in Och because every term is already trivially its own type by [S-Refl].

Well-formedness validates purely structural conditions: annotations are types, binders are in scope, and applications have Π-typed functions with domain-inhabiting arguments. The rules are given in Figure 3.

$$\frac{x \in \mathrm{dom}(\Gamma)}{\Gamma \vdash x \text{ wf}}\,\text{W-Var} \qquad \frac{}{\Gamma \vdash \top \text{ wf}}\,\text{W-Type} \qquad \frac{}{\Gamma \vdash \bot \text{ wf}}\,\text{W-Bot}$$

$$\frac{\Gamma \vdash A \text{ wf} \quad \Gamma, x \leq A \vdash b \text{ wf}}{\Gamma \vdash \lambda(x \leq A).b \text{ wf}}\,\text{W-Lam} \qquad \frac{\Gamma \vdash f \text{ wf} \quad \Gamma \vdash a \text{ wf} \quad \emptyset; \Gamma \vdash f \sqsubseteq \lambda(x \leq A).B \quad \emptyset; \Gamma \vdash a \sqsubseteq A}{\Gamma \vdash f\ a \text{ wf}}\,\text{W-App}$$

$$\frac{\Gamma \vdash A \text{ wf} \quad \Gamma, x \leq A \vdash b \text{ wf}}{\Gamma \vdash \iota(x \leq A).b \text{ wf}}\,\text{W-Iota} \qquad \frac{\Gamma \vdash A \text{ wf} \quad \Gamma, x \leq A \vdash b \text{ wf}}{\Gamma \vdash \mathrm{fix}(x \leq A).b \text{ wf}}\,\text{W-Fix}$$

Figure 3: Well-formedness rules.

[W-Var] requires variables be well-scoped.

[W-Lam], [W-Iota], [W-Fix] check that the annotation is well-formed and that the body is well-formed under the extended context.

[W-App] is the only rule with subtyping premises: the function must subtype some $\lambda(x \leq A).B$ (it has a Π-type), and the argument must subtype the domain A. No type is assigned to the result — the result's type is the result itself (by [S-Refl]), and if the caller needs to know it subtypes something, that is a separate subtyping question.

4.3 Seen set S

Due to the prevelant usage of unbounded recursion in type definitions and encodings, many subtype checks involve infinite proof trees (e.g. $\emptyset; \emptyset \vdash \text{add one two} \sqsubseteq \text{Nat}$).

To derive these infinite trees in finite time Claude said I should use "the Brandt-Henglein device for equirecursive subtyping" as used by Brandt and Henglein (1998). Claude explains it as: "S is a finite representation of a coinductive (potentially infinite) proof tree: when a goal recurs, the branch closes rather than recursing forever, encoding a regular infinite derivation as a finite one."

As far as I can tell the reason it is remarkable enough to have a name is not that using it is particularly hard, but that convincing oneself it is a sound technique is hard, and Brandt/Henglein did just that. I am informed it only works when the cycles it closes contain at least one "productive" step, which are grouped together into Section 4.7.

The "real" subtyping judgment is $\emptyset; \Gamma \vdash a \sqsubseteq b$ (empty hypothesis set); non-empty S arises only inside a derivation.

4.4 Subtyping Rules

The subtyping rules fall into four categories. Each serves a distinct purpose; knowing which category a rule belongs to predicts its shape.

Category	Purpose	Rules	Extends S?
Structural (Section 4.5)	Plumbing: recurse without inspecting constructors on either side	Figure 4	no
Congruence (Section 4.6)	Match constructor on both sides; reduce to sub-obligations with variance	Figure 5	no
Productive unfolding (Section 4.7)	Unfold a recursive binder (ι/fix); extend S; enable coinductive closure	Figure 6	**yes**
Conversion (Section 4.8)	Close under head reduction so subtyping is closed under computation	Figure 7	no

Table 1: Rule taxonomy for declarative subtyping

The S-extension column matters: S-Hyp can only fire against entries that some ancestor productive rule installed, so any path to S-Hyp must cross an unfold — the productivity requirement made mechanical.

4.5 Structural rules

Structural rules (Figure 4) talk about $\sqsubseteq$ as a relation on terms without inspecting either side's head constructor: reflexivity, transitivity, the top element, hypothesis lookup, and variable lookup.

$$\frac{}{S;\Gamma \vdash e \sqsubseteq e}\ \text{S-Refl} \qquad \frac{S;\Gamma \vdash a \sqsubseteq b \quad S;\Gamma \vdash b \sqsubseteq c}{S;\Gamma \vdash a \sqsubseteq c}\ \text{S-Trans} \qquad \frac{(a,b) \in S}{S;\Gamma \vdash a \sqsubseteq b}\ \text{S-Hyp}$$

$$\frac{}{S;\Gamma \vdash x \sqsubseteq \Gamma(x)}\ \text{S-Var} \qquad \frac{}{S;\Gamma \vdash e \sqsubseteq \top}\ \text{S-Top} \qquad \frac{}{S;\Gamma \vdash \bot \sqsubseteq e}\ \text{S-BotL}$$

Figure 4: Structural subtyping rules.

Ex falso via subsumption. There is no dedicated "absurd" eliminator. If $a \sqsubseteq \bot$ is derivable, then $a \sqsubseteq e$ for every e via [S-Trans] on [S-BotL]. The "contradiction" discharge is subsumption alone. This matches the DOT (Amin et al. 2012) tradition: $\bot$ inhabits every type trivially in subtyping, so any term whose type is already $\bot$ flows into any expected type without further ceremony.

[S-Trans] is a constructor, not a derived theorem. Given the interpretation of types as sets of terms, and subtyping as subset over those sets, one would expect transitivity to be provable instead of axiomatic. Eliminating transitivity as a primitive rule (*transitivity elimination*) is highly desirable: it simplifies metatheory and makes the relation syntax-directed. PSS (Hutchins 2010) — a closely related system — did not manage to eliminate transitivity. Pasquale and García-Pérez (Pasquale and García-Pérez 2026) partially succeeded in a continuation of that work, but required switching the entire theory to a Krivine machine.

I have no satisfying answer to whether or not transitivity is eliminatable in Och, but I will try get one.

4.6 Congruence rules

Congruence rules (Figure 5) inspect the head constructor on both sides, *require it to match*, and reduce the goal to sub-obligations on the immediate sub-terms with appropriate variance.

$$\frac{S;\Gamma \vdash A_2 \sqsubseteq A_1 \quad S;\Gamma, x \le A_2 \vdash b_1 \sqsubseteq b_2}{S;\Gamma \vdash \lambda(x \le A_1).b_1 \sqsubseteq \lambda(x \le A_2).b_2}\ \text{S-Lam-Cong} \qquad \frac{S;\Gamma \vdash f_2 \sqsubseteq f_1 \quad S;\Gamma \vdash a_2 \sqsubseteq a_1}{S;\Gamma \vdash a_1 \sqsubseteq a_2}\ \text{S-App-Cong}$$

$$\frac{S;\Gamma \vdash A_1 \sqsubseteq A_2 \quad S;\Gamma, x \le A_2 \vdash B_1 \sqsubseteq B_2}{S;\Gamma \vdash \iota(x \le A_1).B_1 \sqsubseteq \iota(x \le A_2).B_2}\ \text{S-Iota-Cong} \qquad \frac{S;\Gamma \vdash A_1 \sqsubseteq A_2 \quad S;\Gamma, x \le A_2 \vdash b_1 \sqsubseteq b_2}{S;\Gamma \vdash \mathrm{fix}(x \le A_1).b_1 \sqsubseteq \mathrm{fix}(x \le A_2).b_2}\ \text{S-Fix-Cong}$$

Figure 5: Congruence subtyping rules.

The equivalence premise on [S-App-Cong] (both directions on the argument) is necessary because a neutral head can use its argument at any variance, so equivalence is the only sound congruence.

4.7 Productive unfolding (extend S)

A rule is *productive* when it replaces a goal whose head is a recursive binder (ι or fix) with a goal where that binder has been unfolded once. Unfolding makes the term *larger* in syntactic size, so no structural induction can traverse an arbitrary chain of unfolds. Productivity instead provides a *coinductive* handle: once at least one unfold has fired, the original goal is guaranteed to eventually re-appear as an ancestor, at which point [S-Hyp] can close the derivation.

The rules (Figure 6) are the only ones that extend S.

$$\frac{S';\Gamma \vdash a \sqsubseteq A \qquad S';\Gamma \vdash a \sqsubseteq B[x \mapsto a]}{S;\Gamma \vdash a \sqsubseteq \iota(x \le A).B}\ \text{S-Iota-Intro} \qquad \frac{S';\Gamma \vdash B[x \mapsto \iota(x \le A).B] \sqsubseteq c}{S;\Gamma \vdash \iota(x \le A).B \sqsubseteq c}\ \text{S-Unfold-Iota-L}$$

$$\frac{S';\Gamma \vdash a \sqsubseteq B[x \mapsto \iota(x \le A).B]}{S;\Gamma \vdash a \sqsubseteq \iota(x \le A).B}\ \text{S-Unfold-Iota-R}$$

$$\frac{S';\Gamma \vdash b[x \mapsto \mathrm{fix}(x \le A).b] \sqsubseteq c}{S;\Gamma \vdash \mathrm{fix}(x \le A).b \sqsubseteq c}\ \text{S-Unfold-Fix-L} \qquad \frac{S';\Gamma \vdash a \sqsubseteq b[x \mapsto \mathrm{fix}(x \le A).b]}{S;\Gamma \vdash a \sqsubseteq \mathrm{fix}(x \le A).b}\ \text{S-Unfold-Fix-R}$$

Figure 6: Productive unfolding rules. $S' = S, (a, b)$ where $S;\Gamma \vdash a \sqsubseteq b$ is the current goal.

S-Unfold-Iota-R is the weaker sibling of S-Iota-Intro (same conclusion, no annotation premise), needed to close the equivalence of ι-unfolding.

S-Iota-Intro is **the** rule which allows dependent elimitation: it pust the thing being typed **into** the type, allowing the type to depend on the inhabitant.

Worked S-Iota-Intro example: $\underline{\text{true}} \sqsubseteq \underline{\text{DBool}}$. With

$$\underline{\text{true}} \coloneqq \lambda(P \le \top).\lambda(t \le \top).\lambda(f \le \top).t$$
$$\underline{\text{false}} \coloneqq \lambda(P \le \top).\lambda(t \le \top).\lambda(f \le \top).f$$
$$\underline{\text{DBool}} \coloneqq \mathrm{fix}(B \le \top).\iota(\text{self} \le B).\lambda(P \le B \to \top).\lambda(t \le P\ \underline{\text{true}}).\lambda(f \le P\ \underline{\text{false}}).P\ \text{self}$$

the constructors are plain lambdas with $\top$ domains — they carry no recursive reference to $\underline{\text{DBool}}$. The subtyping check $\emptyset;\Gamma \vdash \underline{\text{true}} \sqsubseteq \underline{\text{DBool}}$ still requires the coinductive seen-set to close, because [S-Iota-Intro]'s annotation premise cycles back to the root goal:

$\emptyset;\Gamma \vdash \underline{\text{true}} \sqsubseteq \underline{\text{DBool}}$ — root goal
- $S_1;\Gamma \vdash \underline{\text{true}} \sqsubseteq \iota(\text{self} \le \underline{\text{DBool}}).\lambda(P \le \underline{\text{DBool}} \to \top).\lambda(t \le P\ \underline{\text{true}}).\lambda(f \le P\ \underline{\text{false}}).P\ \ \text{self}$ — S-Unfold-Fix-R
  where $S_1 = \{(\underline{\text{true}}, \underline{\text{DBool}})\}$
  S-Iota-Intro — annotation premise:
  - $S_1;\Gamma \vdash \underline{\text{true}} \sqsubseteq \underline{\text{DBool}}$ — S-Hyp ✓
    root goal reappears — closed by $(\underline{\text{true}}, \underline{\text{DBool}}) \in S_1$
  S-Iota-Intro — body premise (after $[\text{self} \mapsto \underline{\text{true}}]$*):*
  - $S_1;\Gamma \vdash \underline{\text{true}} \sqsubseteq \lambda(P \le \underline{\text{DBool}} \to \top).\lambda(t \le P\ \underline{\text{true}}).\lambda(f \le P\ \underline{\text{false}}).P\ \underline{\text{true}}$ — S-Lam
    contravariant: $(\underline{\text{DBool}} \to \top) \sqsubseteq \top$ *by S-Top ✓*
    covariant body under $P \le \underline{\text{DBool}} \to \top$*:*
    - $S_1;\Gamma \vdash \lambda(t \le \top).\lambda(f \le \top).t \sqsubseteq \lambda(t \le P\ \underline{\text{true}}).\lambda(f \le P\ \underline{\text{false}}).P\ \underline{\text{true}}$ — S-Lam
      contravariant: $P\ \underline{\text{true}} \sqsubseteq \top$ *by S-Top ✓*
      covariant body under $t \le P\ \underline{\text{true}}$*: S-Lam again*
      - *contravariant:* $P\ \underline{\text{false}} \sqsubseteq \top$ *by S-Top ✓*
      - *covariant body:* $t \sqsubseteq P\ \underline{\text{true}}$ *by S-Var* $(t \le P\ \underline{\text{true}} \in \Gamma)$ ✓

Because the constructors have $\top$ domains, every contravariant [S-Lam] premise is discharged by [S-Top]. The only coinductive step is the [S-Iota-Intro] annotation premise, which closes via [S-Hyp] after one productive unfold. This is the Brandt–Henglein discipline in action: recursion in the subtyping judgment is legal *only across a productive unfold*, so non-productive loops (e.g. reflexivity-by-loop) cannot sneak through.

4.8 Conversion

The subtyping relation must be closed under head reduction: if a computes to a', then $a \sqsubseteq b$ should hold iff $a' \sqsubseteq b$. The conversion rules (Figure 7) provide that closure.

$$\frac{S; \Gamma \vdash \text{body}[x \mapsto \text{arg}] \sqsubseteq b}{S; \Gamma \vdash (\lambda(x \leq A).\ \text{body})\ \ \text{arg} \sqsubseteq b}\ \text{S-Beta-L} \qquad \frac{S; \Gamma \vdash a \sqsubseteq \text{body}[x \mapsto \text{arg}]}{S; \Gamma \vdash a \sqsubseteq (\lambda(x \leq A).\ \text{body})\ \ \text{arg}}\ \text{S-Beta-R}$$

Figure 7: Conversion rules.

5 Example Programs

All encodings below are Church/Scott-style: data types are represented as their own eliminators, with no primitive inductive types in the language. Every definition is mechanised and tested in Lean 4 (see Section 7). This section builds from simple encodings to two programs that demonstrate Och's ability to catch subtle bugs in dependently-typed code. Throughout, underlined names (true, Nat, etc.) denote meta-level definitions, as opposed to object-level bound variables (x, n, P, etc.).

5.1 Booleans

$$\begin{aligned} \underline{\text{true}} &:= \lambda\_.\lambda t.\lambda\_.t \\ \underline{\text{false}} &:= \lambda\_.\lambda\_.\lambda f.f \\ \underline{\text{Bool}} &:= \lambda X.\lambda(t \leq X).\lambda(f \leq X).X \end{aligned}$$

```
data Bool : Set where
  true  : Bool
  false : Bool
```

(roughly equivalent Agda)

Bool is a standard Church-encoded boolean: the eliminator takes two branches and a return type X, and the type itself is X. Elimination is non-dependent — the return type X is the same regardless of whether the value is true or false.

$$\begin{aligned} \underline{\text{DBool}} := \text{fix}\ B.\ \iota(\text{self} \leq B). \\ \lambda(P \leq B \to \top).\lambda(t \leq P\ \underline{\text{true}}).\lambda(f \leq P\ \underline{\text{false}}).P\ \text{self} \end{aligned}$$

```
data DBool : Set where
  true  : DBool
  false : DBool
```

(roughly equivalent Agda)

DBool is the *dependent* counterpart: its motive P is a function from values to types, so the return type P self varies with the value being eliminated. This is what ι enables — the self binder in $\iota(\text{self} \leq B)$ refers to the actual value inhabiting the type, and [S-Iota-Intro] substitutes it into P self during type checking.

Crucially, Bool and DBool *share the same constructors* {true, false} (Figure 8). The dependent elimination machinery lives entirely in the type (DBool's ι and motive), not in the constructors. At runtime, a DBool value is identical to a Bool value — the difference is purely compile-time. This pattern recurs later with Fin reusing Nat's constructors (Section 5.4).

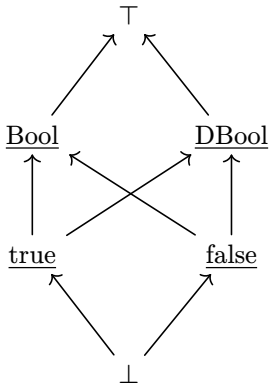

Figure 8: Subtyping lattice for booleans. Arrows denote $\sqsubseteq$.

5.2 Pairs

$$\begin{aligned}
\underline{\text{Pair}} &\coloneqq \lambda A.\lambda B. \\
&\quad \lambda(X \le \top).\lambda(k \le A \to B \to X).X \\
\underline{\text{pair}} &\coloneqq \lambda A.\lambda B.\lambda(a \le A).\lambda(b \le B). \\
&\quad \lambda(X \le \top).\lambda(k \le A \to B \to X).kab
\end{aligned}$$

```
record Pair (A B : Set) : Set where
  constructor pair
  field fst : A; snd : B
```

(roughly equivalent Agda)

Church-encoded binary products. Projection is performed by applying the pair to a continuation that selects the desired component: $\underline{\text{pair}}\ A\ B\ a\ b\ X\ (\lambda(a \le A).\lambda(\_ \le B).a)$ returns a.

5.3 Natural numbers

For natural numbers we use Scott encodings in which a natural is still it's own eliminator, but only eliminates a single level instead of encoding a full fold. This lines up closer to how pattern matching works with standard datatypes in languages like Haskell than Church encodings.

$$\begin{aligned}
\underline{\text{zero}} &\coloneqq \lambda(P \le \top).\lambda(z \le \top).\lambda(s \le \top).z \\
\underline{\text{succ}} &\coloneqq \lambda(n \le \top).\lambda(P \le \top).\lambda(z \le \top). \\
&\quad \lambda(s \le n \to \top).s\ n \\
\underline{\text{Nat}} &\coloneqq \text{fix}(N \le \top).\iota(\text{self} \le N). \\
&\quad \lambda(P \le N \to \top).\lambda(z \le P\ \underline{\text{zero}}). \\
&\quad \lambda(s \le \lambda(n \le N).P\ (\underline{\text{succ}}\ n)). \\
&\quad P\ \text{self}
\end{aligned}$$

```
data Nat : Set where
  zero : Nat
  succ : Nat → Nat
```

(roughly equivalent Agda)

Nat uses both key features of Och. The outer fix ties the recursive knot (Nat refers to itself in the motive domain). The inner ι binds self to the value being typed, enabling *dependent* elimination: the return type P self varies with the value.

One thing to note is the constructors have very wide types compared to Nat:

- zero will return z regardless of it's type, whereas in Nat that z argument is restricted down to P zero.
- succ calls the continuation s with the predecessor n, and therefore only requires that s accepts n (i.e. $\lambda(s \le n \to \top).s\ n$) instead of requiring that it accept any Nat (i.e. $\lambda(s \le \underline{\text{Nat}} \to \top).s\ n$).

Addition is defined with an explicit fix since the Scott-style eliminator provides no induction hypothesis:

$$\begin{aligned}
\underline{\text{add}} &\coloneqq \text{fix}(\text{add} \le \underline{\text{Nat}} \to \underline{\text{Nat}} \to \underline{\text{Nat}}). \\
&\quad \lambda(n \le \underline{\text{Nat}}).\lambda(m \le \underline{\text{Nat}}). \\
&\quad n\ (\underline{\text{Nat}} \to \underline{\text{Nat}}) \\
&\qquad m \\
&\qquad (\lambda(\text{pred} \le \underline{\text{Nat}}). \\
&\qquad\quad \underline{\text{succ}}\ (\text{add pred}\ m))
\end{aligned}$$

```
add : Nat → Nat → Nat
add zero      m = m
add (succ pred) m = succ (add pred m)
```

(roughly equivalent Agda)

5.4 Finite sets

Fin n is the type of naturals strictly less than n. The encoding strategy is: eliminate n to compute the type.

- The zero case yields $\bot$ (uninhabited — there are no naturals less than zero).
- The successor case yields an ι-type which is the same as Nat except it is more precise about which number is passed to the succ case fs.

The overall structure — case-split on the index, $\bot$ for the empty case, ι for the successor case — is shared with Kind (Taelin 2021b), which uses self-types nearly identically to ι (see `base/Fin.kind` in the Kind-Legacy repository). The general machinery for elaborating inductive definitions to lambda-encodings with self-types is developed by (Fu and Stump 2014) and extended to indexed families in Cedille's implementation, though no published work gives an explicit lambda-encoding of Fin specifically.

$$
\begin{aligned}
&\underline{\text{Fin}} \coloneqq \text{fix}(F \leq \underline{\text{Nat}} \to \top).\lambda(n \leq \underline{\text{Nat}}).\\
&\quad n\ (\lambda(\_ \leq \underline{\text{Nat}}).\top)\ \bot\\
&\quad (\lambda(\text{pred} \leq \underline{\text{Nat}}).\\
&\qquad \iota(\text{self} \leq \underline{\text{Nat}}).\\
&\qquad \lambda(P \leq \underline{\text{Nat}} \to \top).\\
&\qquad \lambda(\text{fz} \leq P\ \underline{\text{zero}}).\\
&\qquad \lambda(\text{fs} \leq \lambda(q \leq F\ \ \text{pred}).\\
&\qquad\quad P\ (\underline{\text{succ}}\ q)).\\
&\qquad P\ \ \text{self})
\end{aligned}
$$

```
data Fin : Nat → Set where
  fzero : Fin (succ n)
  fsuc : Fin n → Fin (succ n)
```

(roughly equivalent Agda)

In this encoding self is typed as a Nat, not as a Fin. This is imprecise because in reality the term will always be a Fin, but it's not *wrong* to approximate it as a Nat because $\underline{\text{Fin}}\ n \sqsubseteq \underline{\text{Nat}}$, and difficulties were encountered when trying to type self as a Fin.

Natural number literals inhabit Fin by subsumption: $\underline{\text{zero}} \sqsubseteq \underline{\text{Fin}}\ (\underline{\text{succ}}\ n)$ holds for any n, and $n \sqsubseteq \underline{\text{Fin}}\ n$ is correctly rejected (the diagonal). No separate FZ/FS constructors are needed — the Nat constructors are reused directly, just as Bool and DBool share constructors (Section 5.1). Figure 9 illustrates the subtyping relationships.

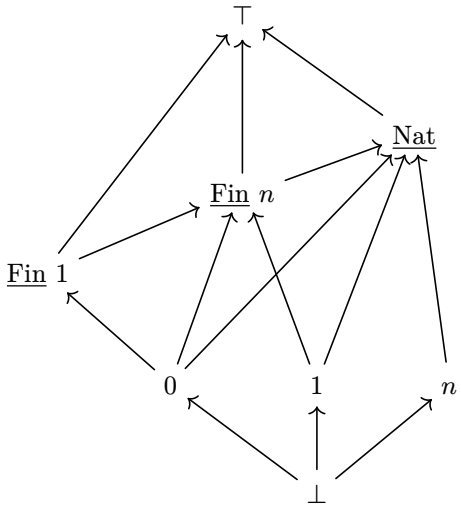

Figure 9: Subtyping lattice for natural numbers and finite sets. Laid out such that subtypes are physically beneath their supertypes.

5.5 Length-indexed arrays

$$
\begin{aligned}
&\underline{\text{Array}} \coloneqq \text{fix}(\text{Arr} \leq \underline{\text{Nat}} \to \top \to \top).\\
&\qquad \lambda(n \leq \underline{\text{Nat}}).\lambda(T \leq \top).\\
&\qquad n\ (\underline{\text{Nat}} \to \top)\\
&\qquad\quad \underline{\text{Unit}}\\
&\qquad\quad (\lambda(\text{pred} \leq \underline{\text{Nat}}).\\
&\qquad\qquad \underline{\text{Pair}}\ T\ (\text{Arr pred}\ T))
\end{aligned}
$$

```
Array : Nat → Set → Set
Array zero     T = Unit
Array (succ n) T = Pair T (Array n T)
```

(roughly equivalent Agda)

$\underline{\text{Array}}\ n\ T$ computes by eliminating the length index: $\underline{\text{Array}}\ \underline{\text{zero}}\ T \equiv \underline{\text{Unit}}$ and $\underline{\text{Array}}\ (\underline{\text{succ}}\ k)\ T \equiv \underline{\text{Pair}}\ T\ \left(\underline{\text{Array}}\ k\ T\right)$. Arrays are built with unit/pair directly.

5.6 Dependent pairs

The dependent pair Sigma packages a witness with a payload whose type depends on the witness:

$$
\begin{aligned}
\underline{\text{Sigma}} \coloneqq\ & \lambda A.\lambda(B \le A \to \top).\\
& \lambda(X \le \top).\\
& \lambda(k \le \lambda(a \le A).B\ a \to X).X\\
\underline{\text{dpair}} \coloneqq\ & \lambda A.\lambda(B \le A \to \top).\\
& \lambda(a \le A).\lambda(b \le B\ a).\\
& \lambda(X \le \top).\\
& \lambda(k \le \lambda(a \le A).B\ a \to X).\\
& k\ a\ b
\end{aligned}
$$

```
record Sigma (A : Set) (B : A → Set)
    : Set where
  constructor dpair
  field fst : A
  field snd : B fst
```

(roughly equivalent Agda)

It is unusual that Σ can be defined in terms of Π in Och instead of Σ needing to be a primitive, further research required to get to the bottom of whether this is a contribution or a symptom of an inconsistency.

appendArrays concatenates two arrays by recursion on the first array's length index:

$$
\begin{aligned}
&\underline{\text{appendArrays}} \coloneqq\\
&\quad \text{fix}(\text{appendArrays} \le \underline{\text{Nat}} \to \underline{\text{Nat}} \to \top \to \top).\\
&\quad \lambda(T \le \top).\lambda(n_1 \le \underline{\text{Nat}}).\lambda(n_2 \le \underline{\text{Nat}}).\\
&\quad \lambda\Big(\text{arr}_1 \le \underline{\text{Array}}\ n_1\ T\Big).\lambda\Big(\text{arr}_2 \le \underline{\text{Array}}\ n_2\ T\Big).\\
&\quad n_1\\
&\qquad \Big(\lambda({n_1}' \le \underline{\text{Nat}}).\underline{\text{Array}}\ {n_1}'\ T \to \underline{\text{Array}}\ (\underline{\text{add}}\ {n_1}'\ n_2)\ T\Big)\\
&\qquad \Big(\lambda\Big({\text{arr}_1}' \le \underline{\text{Array}}\ \underline{\text{zero}}\ T\Big).\text{arr}_2\Big)\\
&\qquad (\lambda(p \le \underline{\text{Nat}}).\lambda\Big({\text{arr}_1}' \le \underline{\text{Array}}\ (\underline{\text{succ}}\ p)\ T\Big).\\
&\qquad\quad {\text{arr}_1}'\ \Big(\underline{\text{Array}}\ (\underline{\text{add}}\ p\ n_2)\ T\Big)\\
&\qquad\qquad (\lambda(\text{hd} \le T).\lambda\Big(\text{tl} \le \underline{\text{Array}}\ p\ T\Big).\\
&\qquad\qquad\quad \underline{\text{pair}}\ T\ \Big(\underline{\text{Array}}\ (\underline{\text{add}}\ p\ n_2)\ T\Big)\\
&\qquad\qquad\qquad \text{hd}\\
&\qquad\qquad\qquad (\text{appendArrays}\ \ T\ p\ n_2\ \ \text{tl}\ \ \text{arr}_2)))\\
&\quad \text{arr}_1
\end{aligned}
$$

```
appendArrays : ∀ {T n₁ n₂}
  → Array n₁ T → Array n₂ T
  → Array (add n₁ n₂) T
appendArrays {n₁ = zero}    _   arr₂ =
  arr₂
appendArrays {n₁ = succ p} arr₁ arr₂ =
  let (hd , tl) = arr₁ in
  (hd , appendArrays tl arr₂)
```

(roughly equivalent Agda)

```
appendArrays : ∀ {T n₁ n₂}
  → Array n₁ T → Array n₂ T
  → Array (add n₁ n₂) T
appendArrays {n₁} arr₁ arr₂ = (
  case n₁ of
    | zero   => λ arr₁' → arr₂
    | succ p => λ arr₁' →
      let (hd , tl) = arr₁' in
      (hd , appendArrays tl arr₂)
) arr₁
```

(a more literal translation)

The zero case returns the second array unchanged. The successor case destructures the first array as a pair, keeps the head, and recursively appends the tail.

One peculiarity with the above definition is that when we eliminate n_1 our motive is $\lambda({n_1}' \le \underline{\text{Nat}}).\underline{\text{Array}}\ {n_1}'\ T \to \underline{\text{Array}}\ (\underline{\text{add}}\ {n_1}'\ n_2)\ T$ (aka "eliminting n_1 will give us a function") instead of the simpler $\lambda({n_1}' \le \underline{\text{Nat}}).\underline{\text{Array}}\ (\underline{\text{add}}\ {n_1}'\ n_2)\ T$ (aka "eliminating n_1 will give us an array"). The reason is that in the arr_1-is-not-empty case ($n_1 = \underline{\text{succ}}\ p$) we need to get the head and tail of arr_1, but the type system does not know statically that arr_1 is non-empty. A Sufficiently Smart Compiler would narrow the type of arr_1 down to $\underline{\text{Array}}\ (\underline{\text{succ}}\ p)\ T$ (a pair), but alas Och is not there yet, so instead we must make a function expecting ${\text{arr}_1}'$ and immediately pass it arr_1. This is reflected in the *second* Agda translation, which has the pattern match expression return a function expecting ${\text{arr}_1}'$ and immediately passes it arr_1.

5.7 Vectors

Vectors package a length with an array of that length:

$$\underline{\text{Vec}}\ T \coloneqq \underline{\text{Sigma}}\ \underline{\text{Nat}}\ \Big(\lambda(n \le \underline{\text{Nat}}).\underline{\text{Array}}\ n\ T\Big)$$

```
Vec : Set → Set
Vec T = Sigma Nat (λ n → Array n T)
```

(roughly equivalent Agda)

The north-star example. appendVec unpacks two vectors, concatenates their arrays with appendArrays, and repacks the result with the summed length using dpair:

$$\begin{aligned}
&\underline{\text{appendVec}} \coloneqq \lambda(T \le \top).\\
&\quad \lambda(v_1 \le \underline{\text{Vec}}\ T).\lambda(v_2 \le \underline{\text{Vec}}\ T).\\
&\quad v_1\ (\underline{\text{Vec}}\ T)\\
&\qquad \Big(\lambda(n_1 \le \underline{\text{Nat}}).\lambda\Big(\text{arr}_1 \le \underline{\text{Array}}\ n_1\ T\Big).\\
&\qquad v_2\ (\underline{\text{Vec}}\ T)\\
&\qquad\quad \Big(\lambda(n_2 \le \underline{\text{Nat}}).\lambda\Big(\text{arr}_2 \le \underline{\text{Array}}\ n_2\ T\Big).\\
&\qquad\quad \underline{\text{dpair}}\ \underline{\text{Nat}}\ \Big(\lambda(n \le \underline{\text{Nat}}).\underline{\text{Array}}\ n\ T\Big)\\
&\qquad\qquad (\underline{\text{add}}\ n_1\ n_2)\\
&\qquad\qquad \Big(\underline{\text{appendArrays}}\ T\ n_1\ n_2\ \text{arr}_1\ \text{arr}_2\Big)\Big)\Big)
\end{aligned}$$

```
appendVec : ∀ {T} → Vec T → Vec T → Vec T
appendVec (n₁ , arr₁) (n₂ , arr₂) =
  (add n₁ n₂ , appendArrays arr₁ arr₂)
```

(roughly equivalent Agda)

This type-checks: the length $\underline{\text{add}}\ n_1\ n_2$ matches the length of the concatenated array.

Now consider a version with a deliberate bug — $\underline{\text{add}}\ n_1\ n_1$ instead of $\underline{\text{add}}\ n_1\ n_2$:

$$\underline{\text{appendVec}}_{\text{wrong}} \coloneqq \ldots\underline{\text{dpair}}\ \underline{\text{Nat}}\ \Big(\lambda(n \le \underline{\text{Nat}}).\underline{\text{Array}}\ n\ T\Big)\ (\underline{\text{add}}\ n_1\ n_1)\ \Big(\underline{\text{appendArrays}}\ T\ n_1\ n_2\ \text{arr}_1\ \text{arr}_2\Big)\ldots$$

The type checker *rejects* this: $\text{arr}_2 \sqsubseteq \underline{\text{Array}}\ n_1\ T$ fails because arr_2 has type $\underline{\text{Array}}\ n_2\ T$ and $n_2 \neq n_1$. The system catches a bug as subtle as adding the wrong two numbers.

5.8 indexArr: safe array indexing

indexArr looks up the i-th element of an $\underline{\text{Array}}\ n\ T$, where i has type $\underline{\text{Fin}}\ n$:

$$\begin{aligned}
&\underline{\text{indexArr}} \coloneqq \text{fix indexArr}.\lambda(T \le \top).\lambda(n \le \underline{\text{Nat}}).\\
&\quad n\ \Big(\lambda(m \le \underline{\text{Nat}}).\underline{\text{Array}}\ m\ T \to \underline{\text{Fin}}\ m \to T\Big)\\
&\qquad \Big(\lambda\Big(\text{arr} \le \underline{\text{Array}}\ \underline{\text{zero}}\ T\Big).\lambda(i \le \underline{\text{Fin}}\ \underline{\text{zero}}).i\Big)\\
&\qquad (\lambda(p \le \underline{\text{Nat}}).\lambda\Big(\text{arr} \le \underline{\text{Array}}\ (\underline{\text{succ}}\ p)\ T\Big).\lambda(i \le \underline{\text{Fin}}\ (\underline{\text{succ}}\ p)).\\
&\qquad\quad \text{arr}\ \ T\ (\lambda(\text{hd} \le T).\lambda\Big(\text{tl} \le \underline{\text{Array}}\ p\ T\Big).\\
&\qquad\qquad i\ (\lambda(_ \le \underline{\text{Nat}}).T)\\
&\qquad\qquad\quad \text{hd}\\
&\qquad\qquad\quad (\lambda(q \le \underline{\text{Fin}}\ p).\ \text{indexArr}\ \ T\ p\ \ \text{tl}\ \ q))
\end{aligned}$$

```
indexArr : ∀ {T n}
  → Array n T → Fin n → T
indexArr {n = zero}  arr i =
  absurd i
indexArr {n = succ p} (hd ,
tl) fzero =
  hd
indexArr {n = succ p} (hd ,
tl) (fsucc j) =
  indexArr tl j
```

(roughly equivalent Agda)

In the zero-length branch, i has type $\underline{\text{Fin}}\ \underline{\text{zero}} = \bot$, so the branch body is i itself — ex falso via [S-BotL]. In the successor branch, the array is destructured as a pair and i eliminates to either the head or a recursive call on the tail.

The payoff is at call sites:

$$\begin{aligned}
\underline{\text{indexArr}}\ \underline{\text{Nat}}\ \underline{\text{three}}\ \ \text{arr}\ \ \underline{\text{zero}}\ \ &\checkmark\ \ (\underline{\text{zero}} \sqsubseteq \underline{\text{Fin}}\ \underline{\text{three}})\\
\underline{\text{indexArr}}\ \underline{\text{Nat}}\ \underline{\text{three}}\ \ \text{arr}\ \ \underline{\text{two}}\ \ &\checkmark\ \ (\underline{\text{two}} \sqsubseteq \underline{\text{Fin}}\ \underline{\text{three}})\\
\underline{\text{indexArr}}\ \underline{\text{Nat}}\ \underline{\text{three}}\ \ \text{arr}\ \ \underline{\text{three}}\ \ &\times\ \ (\underline{\text{three}} \not\sqsubseteq \underline{\text{Fin}}\ \underline{\text{three}})
\end{aligned}$$

Out-of-bounds access is a *compile-time* error: the index literal fails to subtype $\underline{\text{Fin}}\ n$, and no runtime check is needed.

5.9 Eater: concrete evaluation with fix

The following example demonstrates concrete evaluation of a recursive term defined via fix.

$$\underline{\text{bool}} := \lambda t.\lambda f.\top$$
$$\underline{\text{true}} := \lambda t.\lambda f.t$$
$$\underline{\text{false}} := \lambda t.\lambda f.f$$
$$\underline{\text{eater}} := \text{fix } e.\lambda(b \leq \underline{\text{bool}}).b\top e$$

$\underline{\text{eater}}$ is a recursive function that consumes $\underline{\text{false}}$ arguments until it hits a $\underline{\text{true}}$, at which point it returns $\top$. At each step, fix unrolls via [E-Fix], and the boolean argument selects between $\top$ (stop) and the self-reference e (continue). E.g. $\underline{\text{eater}}\ \underline{\text{true}} \Downarrow \top$ and similarly $\underline{\text{eater}}\ \underline{\text{false}}\ \underline{\text{false}}\ \underline{\text{false}}\ \underline{\text{true}} \Downarrow \top$.

Evaluation of $\underline{\text{eater}}\ \underline{\text{false}}\ \underline{\text{true}}$. We abbreviate $\underline{\text{eater}}$'s unrolled form $\lambda(b \leq \underline{\text{bool}}).b\top\ \underline{\text{eater}}$ as $\underline{\text{eater}}^\circ$ for readability.

$\underline{\text{eater}}\ \underline{\text{false}}\ \underline{\text{true}} \Downarrow \top$	E-App
$\quad \underline{\text{eater}}\ \underline{\text{false}} \Downarrow \underline{\text{eater}}^\circ$	E-App
$\quad\quad \underline{\text{eater}} \Downarrow \underline{\text{eater}}^\circ$	E-Fix
$\quad\quad \underline{\text{false}} \Downarrow \underline{\text{false}}$	E-Val
$\quad\quad (b\top\ \underline{\text{eater}})[b \mapsto \underline{\text{false}}] \Downarrow \underline{\text{eater}}^\circ$	$\underline{\text{false}}$ selects second arg
$\quad \underline{\text{true}} \Downarrow \underline{\text{true}}$	E-Val
$\quad (b\top\ \underline{\text{eater}})[b \mapsto \underline{\text{true}}] \Downarrow \top$	$\underline{\text{true}}$ selects first arg ✓

Each [E-Fix] step replaces the bound variable e with the entire fix expression, producing a λ that is ready to accept the next argument. The recursion terminates when $\underline{\text{true}}$ selects $\top$ instead of the self-reference.

Evaluation of $\underline{\text{eater}}\ \underline{\text{true}}\ \underline{\text{false}}$ **(explodes):**

$\underline{\text{eater}}\ \underline{\text{true}}\ \underline{\text{false}} \Downarrow ?$	E-App
$\quad \underline{\text{eater}}\ \underline{\text{true}} \Downarrow \top$	as above — $\underline{\text{true}}$ selects first arg
$\quad \underline{\text{false}} \Downarrow \underline{\text{false}}$	E-Val
$\quad \top\ \underline{\text{false}} \Downarrow ?$	**stuck** — $\top$ is not a λ ×

There is no rule whose conclusion matches $\top\ \underline{\text{false}} \Downarrow v$: [E-App] requires the function position to evaluate to a λ, and $\top$ is not one. The derivation is stuck — this is exactly the kind of runtime error the type system is designed to prevent.

6 Metatheory

(This section is a stub, significant attempts at soundness have been made but have so far only resulted in proofs of un-soundness.)

Soundness connects the concrete semantics (Section 3.1), subtyping (Section 4.4), and well-formedness (Section 4.2).

Evaluation equivalence. If e evaluates to e', both directions of subtyping hold.

$$e \Downarrow e' \text{ implies } \emptyset;\emptyset \vdash e' \sqsubseteq e \text{ and } \emptyset;\emptyset \vdash e \sqsubseteq e'$$

Progress. A well-formed closed term doesn't get stuck during evaluation. The evaluator may return a value or run out of fuel but never reaches an error state.

End-to-end. Composing the above: if e is well-formed and subtypes τ, and e evaluates to e', the result subtypes τ.

$$\emptyset \vdash e \text{ wf and } \emptyset;\emptyset \vdash e \sqsubseteq \tau \text{ and } e \Downarrow e' \text{ implies } \emptyset;\emptyset \vdash e' \sqsubseteq \tau$$

6.1 What soundness promises to the programmer

Two separable runtime properties a type system can promise:

1. **Termination.** Running a well-typed program eventually produces a value. A totality / normalization claim.
2. **No runtime type errors.** Conditional on the program *not* diverging, the value it produces is well-formed and the program never reaches a stuck state.

Och **does not aim to prove (1).** Consistency and normalization are explicitly deferred; the calculus admits non-terminating terms by design (type-in-type; general recursion via fix).

Och **does aim to prove (2)**, and this is the real runtime guarantee of the type system. The language expects the following discipline: **type-check first; only evaluate if it succeeded.**

7 Lean Formalisation

The calculus described above has been formalised in Lean 4, which is used to maintain a test suite of example programs and allow AI agents to iterate on the metatheory. The source is available at github.com/charlielidbury/ochre.

8 Related Work

Large-scale verification projects have demonstrated that formal verification of systems software is feasible: CompCert (Leroy 2009) is a verified C compiler (written in Coq, extracted to OCaml), seL4 (Klein et al. 2009) is a verified OS microkernel (C code with a separate Isabelle/HOL refinement proof), and Project Everest (Bhargavan et al. 2017) produced a verified HTTPS stack (F*, extracted to C via KReMLin). These projects use a wide variety of verification techniques, which we overview in this section, with the most relevant/directly comparible to Ochre first.

8.1 Dependently typed systems languages

Low* (the systems fragment of F* (Protzenko et al. 2017)) and ATS (Xi 2017) are the closest prior art to Ochre's vision: languages which combine dependent types with low-level control in a single system. Low* is by far the more widely used of the two — it is the language behind HACL* (Zinzindohoué et al. 2017) and Project Everest (Bhargavan et al. 2017) — so we illustrate with it.

```
(* Low*: safe buffer indexing (systems fragment of F*) *)
val index (#a:Type) (b:buffer a) (i:U32.t)
  : Stack a
    (requires fun h -> live h b /\ U32.v i < length b)
    (ensures  fun h0 r h1 ->
      h0 == h1 /\ r == Seq.index (as_seq h0 b) (U32.v i))

let index #a b i = b.(i)
```

Listing 1: Safe buffer indexing in Low*. The refinement `U32.v i < length b` in the `requires` clause is a compile-time proof obligation, discharged by F*'s SMT backend; callers must prove the index is in bounds. The liveness precondition `live h b` and the heap `h` are threaded through the `Stack` effect — mutation lives in a monadic state effect rather than in the term language. After verification, KaRaMeL extracts `b.(i)` to a bare C array access `b[i]` with no runtime bounds check.

Low* achieves statically guaranteed bounds safety with zero runtime overhead, but inherits F*'s purely functional programming model: mutation is expressed via a monadic state effect (the `Stack`/heap-passing discipline above), not via direct mutable references as in Rust, and the programmer must thread heap invariants and buffer-liveness obligations through that effect by hand. ATS reaches comparable guarantees by a different route — a bespoke constraint language of dependent sorts (the `{i:nat | i < n}` annotations) discharged by its own constraint solver, and linear types rather than an effect for safe mutation — but at the cost of a sort language that is syntactically and semantically separated from the term language.

Ochre aims to differ from both by adopting Rust's ownership model as the mechanism for safe mutation, rather than monadic effects (Low*). The hypothesis is that ownership is a more natural fit for systems

programmers, since Rust has demonstrated that the model is learnable at scale, and that a unified language reduces the cognitive overhead of switching between specification and implementation.

8.2 Parallel Codebases

Aeneas (Ho and Protzenko 2022) takes an existing Rust program, translates it to a pure functional model in a theorem prover (Lean, Coq, F*, or HOL4), and lets the programmer verify properties of the model. The programmer is left with two version of their codebase: the executable artifact and the verified artifact. This decouples the two languages, allowing them the best tool for the job (in Aeneas' case, Rust for execution and Lean for verification).

The key insight is that Rust's ownership discipline guarantees that each mutable borrow has exclusive access, so an in-place mutation `x[i] = v` can be modelled as a pure functional update that returns a new collection.

```rust
// Rust source
pub fn zero(x: &mut Vec<u32>) {
    let mut i = 0;
    while i < x.len() {
        x[i] = 0;
        i += 1;
    }
}
```

```
-- Aeneas-generated Lean
def zero_loop (x : Vec U32) (i : Usize)
  : Result (Vec U32) :=
  if i < Vec.len x then do
    let (_, back) ←
      Vec.index_mut x i
    let x1 := back 0#u32
    zero_loop x1 (i + 1#usize)
  else Result.ok x
```

Listing 2: Aeneas translation of in-place vector zeroing. The `&mut Vec<u32>` becomes a value that is returned; `x[i] = 0` becomes a call to `index_mut` returning a backward continuation `back` which, applied to the new value, produces the updated vector. Failable operations (indexing, arithmetic) live inside the `Result` monad.

This avoids the need to write efficient code in a theorem prover, or reason about the correctness of imperative Rust code. It is the verification technique chosen for Signal Shot (Moura 2026), an initiative to formally verify the Signal protocol's Rust implementation (`libsignal`) in Lean by translating it with Aeneas.

The downside of this two-copies-of-the-code approach is that the programmer must maintain and understand two versions of the codebase in two languages with different semantics. This might involve a proof engineer tweaking the Rust so different Lean is emitted which is more ameenable to the particular proof they are attempting. This is a messy workaround — it would be cleaner if it was all under a single language.

8.3 Bolt-on verification for Rust

Prusti (Astrauskas et al. 2022), Creusot (Denis et al. 2022), and Verus (Lattuada et al. 2023) add specification and verification capabilities to Rust via annotations, macros, or embedded DSLs. The programmer writes Rust code as normal and adds pre/postconditions and loop invariants; an automated verifier (typically backed by an SMT solver) checks them. These tools verify only *safe* Rust; Gillian-Rust (Ayoun et al. 2025) complements Creusot by verifying the type safety and functional correctness of `unsafe` Rust through separation-logic symbolic execution (embedding the lifetime logic of RustBelt (Jung et al. 2018) and the prophecies of RustHornBelt (Matsushita et al. 2022)), discharging exactly the unsafe-code specifications Creusot can express but not verify.

```
// Verus: safe array indexing
verus! {
fn get(v: &Vec<u64>, i: usize) -> (r: u64)
    requires i < v.len(),
    ensures  r == v@[i as int],
{
    v[i]
}
} // verus!
```

Listing 3: Safe array indexing in Verus. The precondition `i < v.len()` is a first-order proof obligation discharged by Verus's SMT backend; every caller must prove the index in bounds, after which the access is statically guaranteed safe (and the runtime bounds check can be elided via a trusted `get_unchecked` primitive, Lattuada et al. (2023) §3. The postcondition relates the result to a ghost `Seq<u64>` model `v@`, since Rust's native `Vec<u64>` cannot appear in logical formulas.

As a demonstration of quite how ergonomic Verus is, Listing 4 gives an entire `binary_search` algorithm verified in Verus — the sortedness and membership preconditions, the loop invariant, and the `decreases` termination measure all sit inline as annotations on otherwise-ordinary Rust, and each `v[mid]` access is proven in bounds from the invariant.

```
// Verus: verified binary search (from verus-lang/verus examples)
verus! {
fn binary_search(v: &Vec<u64>, k: u64) -> (r: usize)
    requires
        forall|i: int, j: int|
            0 <= i <= j < v.len() ==> v[i] <= v[j],
        exists|i: int| 0 <= i < v.len() && k == v[i],
    ensures
        r < v.len(),
        k == v[r as int],
{
    let mut lo: usize = 0;
    let mut hi: usize = v.len() - 1;
    while lo != hi
        invariant
            hi < v.len(),
            exists|i: int| lo <= i <= hi && k == v[i],
            forall|i: int, j: int|
                0 <= i <= j < v.len() ==> v[i] <= v[j],
        decreases hi - lo,
    {
        let mid = lo + (hi - lo) / 2;
        if v[mid] < k { lo = mid + 1; } else { hi = mid; }
    }
    lo
}
} // verus!
```

Listing 4: Verified binary search in Verus. Specifications are written in a first-order logic DSL inside `requires`/`ensures`/`invariant` blocks. Each `v[mid]` access is verified to be in bounds from the loop invariant.

These tools have the lowest adoption barrier: the programmer stays in Rust and adds specifications incrementally. However, specifications operate over ghost models (`Seq<T>`, accessed via `v@`) since Rust's native types cannot appear in logical formulas. The expressivity of specifications is limited by what the SMT backend can decide, and the verification is incomplete — the solver may time out or fail to find a proof, requiring manual intervention in the form of assertions or lemma calls. In a language with native dependent types, by contrast, the precondition $i < n$ is the type of the index argument, not a side-channel annotation, and the type checker itself is the verification engine.

The expressivity ceiling, made concrete. It is worth asking precisely how far this bolt-on approach can reach, because the answer delimits what a unified dependently-typed language buys over it. A potential test case is Project Everest (Bhargavan et al. 2017): could its verified HTTPS stack — built in F*/Low* (Protzenko et al. 2017) — be rebuilt in Verus instead? A large fraction of Everest is exactly Verus's strength: the functional correctness of imperative byte-manipulation code (record-layer formatting, parsers, the handshake state machines). For this class Verus would likely fare *better* than F*, because Rust's borrow checker discharges the aliasing reasoning that is Everest's worst accidental complexity, rather than requiring a separation-logic memory model to be threaded through by hand. But two of Everest's deliverables lie structurally beyond Verus. First, HACL* (Zinzindohoué et al. 2017) guarantees *secret independence* (constant-time execution): branching and memory-access patterns must not depend on secret data. This is a *relational* (2-safety) property — it quantifies over *pairs* of executions that differ only in secrets and asserts their observable traces coincide — and Low* discharges it by parametricity over an abstract secret-integer type, preserved through extraction to C (Protzenko et al. 2017). Verus generates its verification condition by weakest-precondition reasoning over a *single* execution and discharges it with an SMT solver; a single-trace logic cannot so much as *state* a 2-safety property without an explicit product-program encoding, and Verus offers no secret/public type discipline for it. Second, Vale (Fromherz et al. 2019) verifies hand-optimised cryptographic *assembly* using a verified verification-condition generator implemented by proof-by-reflection over F*'s dependent types; Verus has no machine model and, lacking dependent types, tactics, or explicit proof terms, no analogue of the proof-structuring machinery the task relies on.

Tellingly, Verus's authors draw exactly this line themselves: Verus is "closely tied to Rust's type system, which is more limited in some ways than the dependent type systems of Coq and F*[, which] may preclude some more sophisticated styles of structuring proofs," and — in contrast to the engineering limitations they expect to lift — "limitations tied to Rust's type system are imposed by Verus's design choices" (Lattuada et al. 2023). The leading "stay in Rust" verifier thus trades away dependent-type proof power for ergonomics, by construction. Ochre's bet is that unifying the term and type languages need not force that trade: when the specification *is* the type and the type checker *is* the verifier, the proof power of dependent types and the ergonomics of working in one language are no longer in tension. Whether Och's subtyping can be made expressive/sound enough to deliver on that bet in practice is exactly the metatheoretic question this paper opens (Section 6).

8.4 Refinement Types

Refinement types augment a base type with a logical predicate that constrains its inhabitants — an in-bounds index, for instance, has the refinement type `{v: Nat | v < n}` — and discharge the resulting subtyping obligations automatically with an SMT solver. Liquid Haskell (Vazou et al. 2014) is the canonical example, and Flux (Lehmann et al. 2023) brings the same liquid-types discipline to Rust, attaching refinements to Rust's types and checking them alongside the ownership information the borrow checker already provides. Safe array indexing is the textbook application: the index is refined to be smaller than the length, and the resulting obligation is dispatched by the solver.

The defining design choice is that refinement predicates are drawn from a *decidable* logic, which is what keeps checking fully automatic. This is precisely a restriction relative to dependent types: the predicate language is deliberately weaker than the term language, so specifications that require quantifier reasoning, induction, or arbitrary value-dependent computation — which dependent types express directly, as the encodings of Section 5 illustrate — lie outside what refinement types can state or check without escaping to manual proof. Och sits on the dependently-typed side of this trade-off: its types are arbitrary terms rather than predicates in a fixed decidable theory, buying expressive power at the (here, still open) cost of decidable checking.

Bibliography

Amin, Nada, Adriaan Moors, and Martin Odersky. 2012. "Dependent Object Types." In "Workshop on Foundations of Object-Oriented Languages (Fool)." Special issue, *Workshop on Foundations of*

Object-Oriented Languages (FOOL) (Tucson, AZ, USA),. https://infoscience.epfl.ch/server/api/core/bitstreams/678b1818-06aa-47f1-b7e0-783413d76044/content.

Astrauskas, Vytautas, Aurel Bílý, Jonás Fiala, et al. 2022. "The Prusti Project: Formal Verification for Rust." In "NASA Formal Methods (Nfm)." Special issue, *NASA Formal Methods (NFM)*,. https://doi.org/10.1007/978-3-031-06773-0_5.

Ayoun, Sacha-Élie, Xavier Denis, Petar Maksimović, and Philippa Gardner. 2025. "A Hybrid Approach to Semi-Automated Rust Verification." In "Proceedings of the ACM on Programming Languages (Pldi)." Special issue, *Proceedings of the ACM on Programming Languages (PLDI)* 9. https://doi.org/10.1145/3729289.

Bhargavan, Karthikeyan, Barry Bond, Antoine Delignat-Lavaud, et al. 2017. "Everest: Towards a Verified, Drop-in Replacement of Https." In "2nd Summit on Advances in Programming Languages (Snapl)." Special issue, *2nd Summit on Advances in Programming Languages (SNAPL)*,. https://doi.org/10.4230/LIPIcs.SNAPL.2017.1.

Brandt, Michael, and Fritz Henglein. 1998. "Coinductive Axiomatization of Recursive Type Equality and Subtyping." *Fundamenta Informaticae* 33 (4): 309–38. https://doi.org/10.1007/3-540-62688-3_29.

Denis, Xavier, Jacques-Henri Jourdan, and Claude Marché. 2022. "Creusot: A Foundry for the Deductive Verification of Rust Programs." In "Formal Methods and Software Engineering (Icfem)." Special issue, *Formal Methods and Software Engineering (ICFEM)*,. https://doi.org/10.1007/978-3-031-17244-1_6.

Fromherz, Aymeric, Nick Giannarakis, Chris Hawblitzel, Bryan Parno, Aseem Rastogi, and Nikhil Swamy. 2019. "A Verified, Efficient Embedding of a Verifiable Assembly Language." In "Proceedings of the ACM on Programming Languages (Popl)." Special issue, *Proceedings of the ACM on Programming Languages (POPL)* 3. https://doi.org/10.1145/3290376.

Fu, Peng, and Aaron Stump. 2014. "Self Types for Dependently Typed Lambda Encodings." In "Rewriting and Typed Lambda Calculi (RTA-Tlca)." Special issue, *Rewriting and Typed Lambda Calculi (RTA-TLCA)*,. https://homepage.divms.uiowa.edu/~astump/papers/fu-stump-rta-tlca-14.pdf.

Ho, Son, and Jonathan Protzenko. 2022. "Aeneas: Rust Verification by Functional Translation." In "Proceedings of the ACM on Programming Languages (Icfp)." Special issue, *Proceedings of the ACM on Programming Languages (ICFP)* 6. https://doi.org/10.1145/3547647.

Hutchins, DeLesley S. 2010. "Pure Subtype Systems." In "Proceedings of the 37th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (Popl)." Special issue, *Proceedings of the 37th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, 287–98. https://doi.org/10.1145/1706299.1706334.

Jung, Ralf, Jacques-Henri Jourdan, Robbert Krebbers, and Derek Dreyer. 2018. "RustBelt: Securing the Foundations of the Rust Programming Language." In "Proceedings of the ACM on Programming Languages (Popl)." Special issue, *Proceedings of the ACM on Programming Languages (POPL)* 2. https://doi.org/10.1145/3158154.

Klein, Gerwin, Kevin Elphinstone, Gernot Heiser, et al. 2009. "seL4: Formal Verification of an OS Kernel." In "Proceedings of the 22nd ACM Symposium on Operating Systems Principles (Sosp)." Special issue, *Proceedings of the 22nd ACM Symposium on Operating Systems Principles (SOSP)*, 207–20. https://doi.org/10.1145/1629575.1629596.

Knop, Dirk. 2026. "As Many Firefox Vulnerabilities Closed in April as in the Previous Two Years." May. https://www.heise.de/en/news/As-many-Firefox-vulnerabilities-closed-in-April-as-in-the-previous-two-years-11287175.html.

Lattuada, Andrea, Travis Hance, Chanhee Cho, et al. 2023. "Verus: Verifying Rust Programs Using Linear Ghost Types." In "Proceedings of the ACM on Programming Languages (Oopsla)." Special issue, *Proceedings of the ACM on Programming Languages (OOPSLA)* 7. https://doi.org/10.1145/3586037.

Lehmann, Nico, Adam T. Geller, Niki Vazou, and Ranjit Jhala. 2023. "Flux: Liquid Types for Rust." In "Proceedings of the ACM on Programming Languages (Pldi)." Special issue, *Proceedings of the ACM on Programming Languages (PLDI)* 7. https://doi.org/10.1145/3591283.

Leroy, Xavier. 2009. "Formal Verification of a Realistic Compiler." *Communications of the ACM* 52 (7): 107–15. https://doi.org/10.1145/1538788.1538814.

Lidbury, Charlie. 2024. "Ochre: A Dependently Typed Systems Programming Language." MEng Individual Project. https://assets.super.so/f890b173-9aa0-4184-8dbd-d0ee94de3ebb/files/00463014-8f18-41dc-81f5-2dccbb91ceae/Ochre_Thesis_Main.pdf.

Matsushita, Yusuke, Xavier Denis, Jacques-Henri Jourdan, and Derek Dreyer. 2022. "RustHornBelt: A Semantic Foundation for Functional Verification of Rust Programs with Unsafe Code." In "Proceedings of the 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation (Pldi)." Special issue, *Proceedings of the 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation (PLDI)*,. https://doi.org/10.1145/3519939.3523704.

Microsoft Security Response Center. 2026. "CVE-2026-31431: COPY_FAIL Vulnerability Enables Linux Root Privilege Escalation." May. https://www.microsoft.com/en-us/security/blog/2026/05/01/cve-2026-31431-copy-fail-vulnerability-enables-linux-root-privilege-escalation/.

Moura, Leonardo de. 2026. "Signal Shot: The Platform Is Ready." April. https://leodemoura.github.io/blog/2026-4-20-signal-shot-the-platform-is-ready/.

Pasquale, Valentin, and Álvaro García-Pérez. 2026. "Towards the Type Safety of Pure Subtype Systems." In "34th EACSL Annual Conference on Computer Science Logic (Csl)." Special issue, *34th EACSL Annual Conference on Computer Science Logic (CSL)*, Leibniz International Proceedings in Informatics (LIPIcs), vol. 363 : 37:1–16. https://doi.org/10.4230/LIPIcs.CSL.2026.37.

Protzenko, Jonathan, Jean-Karim Zinzindohoué, Aseem Rastogi, et al. 2017. "Verified Low-Level Programming Embedded in F*." In "Proceedings of the ACM on Programming Languages (Icfp)." Special issue, *Proceedings of the ACM on Programming Languages (ICFP)* 1. https://doi.org/10.1145/3110261.

Sumner, Jarred. 2026. "Rewrite Bun from Zig to Rust.". https://github.com/oven-sh/bun/pull/30412.

Taelin, Victor. 2021a. "Beyond Inductive Datatypes: Exploring Self Types.". https://github.com/HigherOrderCO/Kind/blob/5a6a5e2f2cff35f77176009dfb4c2f1a1bac2799/blog/1-beyond-inductive-datatypes.md.

Taelin, Victor. 2021b. "Kind: A Modern Proof Language.". https://github.com/HigherOrderCO-archive/Kind-Legacy/blob/5a6a5e2/base/Fin.kind.

Vazou, Niki, Eric L. Seidel, Ranjit Jhala, Dimitrios Vytiniotis, and Simon Peyton Jones. 2014. "Refinement Types for Haskell." In "Proceedings of the 19th ACM SIGPLAN International Conference on Functional Programming (Icfp)." Special issue, *Proceedings of the 19th ACM SIGPLAN International Conference on Functional Programming (ICFP)*,. https://doi.org/10.1145/2628136.2628161.

Xi, Hongwei. 2017. "Applied Type System: An Approach to Practical Programming with Theorem-Proving." In "Journal of Functional Programming." Special issue, *Journal of Functional Programming*,. https://www.cs.bu.edu/~hwxi/academic/papers/JFPats.pdf.

Zinzindohoué, Jean-Karim, Karthikeyan Bhargavan, Jonathan Protzenko, and Benjamin Beurdouche. 2017. "HACL*: A Verified Modern Cryptographic Library." In "Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (Ccs)." Special issue, *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 1789–806. https://doi.org/10.1145/3133956.3134043.